

TLS 1.3 Key Derivation in QUIC

How Secrets Are Built, Step by Step

With Concrete Byte-Level Examples at Every Stage

And How We Export/Import Them for Server Migration

Reference Document • June 2026

This document explains exactly how QUIC derives its encryption keys from the TLS 1.3 handshake, with concrete hex examples at every step. It covers the three phases of key derivation (Initial, Handshake, Application), explains HKDF in detail, shows what our server migration exports (445 bytes), and demonstrates why the preferred server can reconstruct identical keys from just the traffic secret.

1. The Big Picture: Three Phases of Keys

A QUIC connection goes through three distinct encryption phases. Each phase uses different keys, derived from different inputs. Understanding this progression is essential to understanding what we export for migration.

Phase	Epoch	Key Source	Security	Who Can Decrypt?
1. Initial	0	Destination Connection ID (public)	None (keys are public)	Anyone who sees the DCID
2. Handshake	2	TLS key exchange (ECDHE shared secret)	Forward-secret	Only client and server
3. Application	3+	Derived from master secret	Forward-secret + updatable	Only client and server

For server migration, we only need to export Phase 3 (Application) keys. Phase 1 and 2 are only used during the handshake, which is already complete when migration happens.

The complete derivation chain looks like this:

```
Phase 1: DCID (public)
  ■■■ HKDF-Extract ■■→ initial_secret
    ■■■ HKDF-Expand("client in") ■→ client Initial keys
    ■■■ HKDF-Expand("server in") ■→ server Initial keys

Phase 2: ECDHE shared secret (from key exchange)
  ■■■ HKDF-Extract ■■→ handshake_secret
    ■■■ HKDF-Expand("c hs traffic") ■→ client Handshake keys
    ■■■ HKDF-Expand("s hs traffic") ■→ server Handshake keys

Phase 3: Master secret (from handshake secret)
  ■■■ HKDF-Extract ■■→ master_secret
    ■■■ HKDF-Expand("c ap traffic") ■→ client App secret ← WE EXPORT THIS
    ■■■ HKDF-Expand("s ap traffic") ■→ server App secret ← WE EXPORT THIS
      ■■■ HKDF-Expand("quic key") ■→ AEAD key (16 bytes)
      ■■■ HKDF-Expand("quic iv") ■→ IV (12 bytes)
      ■■■ HKDF-Expand("quic hp") ■→ HP key (16 bytes)
```

2. HKDF: The Building Block of Everything

HKDF (HMAC-based Key Derivation Function, RFC 5869) is the single most important function in TLS 1.3. Every key in QUIC is derived using HKDF. It has two operations:

2.1 HKDF-Extract: Concentrate Randomness

Takes potentially weak input key material (IKM) and a salt, produces a fixed-size pseudorandom key (PRK). Think of it as "distilling" randomness.

```
HKDF-Extract(salt, IKM) → PRK

Internally: PRK = HMAC-SHA256(key=salt, data=IKM)

Example (Initial keys):
salt = 0x38762cf7f55934b34d179ae6a4c80cadccbb7f0a (QUIC v1 Initial salt)
IKM  = 0x8394c8f03e515708 (Destination Connection ID)
      ████████████████████████████████████████
PRK  = 0x7db5df06e7a69e432496adedb0085192
      3595221596ae2ae9fb8115c1e9ed0a44 (32 bytes = initial_secret)
```

The salt is a fixed value defined by each QUIC version. For QUIC v1, it's a 20-byte constant from RFC 9001 Section 5.2. The IKM is the Destination Connection ID from the client's first packet -- which is sent in the clear!

This is why Initial packets offer NO real confidentiality. Anyone who reads the DCID from the wire can compute the exact same initial_secret and decrypt everything.

2.2 HKDF-Expand-Label: Derive Specific Keys

Takes a PRK and a label string, produces a key of any desired length. This is how one secret becomes many different keys.

```
HKDF-Expand-Label(Secret, Label, Context, Length) → Key

Internally:
1. Build HkdfLabel struct:
   length = desired output length (2 bytes)
   label  = "tls13 " + Label (variable, with length prefix)
   context = Context (variable, with length prefix, usually empty)

2. Key = HKDF-Expand(PRK=Secret, info=HkdfLabel, L=Length)
      = HMAC-SHA256(key=Secret, data=HkdfLabel || 0x01) [for first block]

Example (deriving client Initial secret):
Secret = initial_secret (32 bytes from Extract above)
Label  = "client in" → prefixed as "tls13 client in"
Context = "" (empty)
Length  = 32
      ████████████████████████████████████████
Output  = 0xc00cf151ca5be075ed0ebfb5c80323c4
      2d6b7db67881289af4008f1f6c357aea (client_initial_secret)
```

The label acts as a domain separator -- even with the same secret, different labels produce completely different keys. This is how "quic key", "quic iv", and "quic hp" all derive different values from the same traffic secret.

3. Phase 1: Initial Keys (Epoch 0)

The very first QUIC packet must be encrypted, but no key exchange has happened yet. Solution: derive keys from the Destination Connection ID, which both sides know.

Step-by-Step with Real Values (RFC 9001 Appendix A)

These are the exact test vectors from the RFC. You can verify them yourself.

Step 1: Client picks a random DCID

```
Client's Initial packet:  
Destination Connection ID = 0x8394c8f03e515708 (8 bytes, random)
```

This is in the clear -- anyone sniffing the network sees this value.

Step 2: Extract the initial secret

```
initial_secret = HKDF-Extract(  
    salt = 0x38762cf7f55934b34d179ae6a4c80cadccbb7f0a, // QUIC v1 fixed salt  
    IKM  = 0x8394c8f03e515708                        // DCID from wire  
)  
  
Result: 0x7db5df06e7a69e432496adedb00851929595221596ae2ae9fb8115c1e9ed0a44
```

Step 3: Derive client and server Initial secrets

```
client_initial_secret = HKDF-Expand-Label(  
    Secret = initial_secret,  
    Label  = "client in",    // TLS 1.3 label  
    Length = 32  
)  
= 0xc00cf151ca5be075ed0ebfb5c80323c42d6b7db67881289af4008f1f6c357aea  
  
server_initial_secret = HKDF-Expand-Label(  
    Secret = initial_secret,  
    Label  = "server in",  
    Length = 32  
)  
= 0x3c199828fd139efd216c155ad844cc81fb82fa8d7446fa7d78be803acdda951b
```

Step 4: From each secret, derive AEAD key + IV + HP key

```
From client_initial_secret:
  ■■ HKDF-Expand-Label(Label="quic key", Length=16)
    = 0x1f369613dd76d5467730efcbe3b1a22d // AES-128 key (16 bytes)
  ■■ HKDF-Expand-Label(Label="quic iv", Length=12)
    = 0xfa044b2f42a3fd3b46fb255c // IV / nonce (12 bytes)
  ■■ HKDF-Expand-Label(Label="quic hp", Length=16)
    = 0x9f50449e04a0e810283a1e9933adedd2 // Header Protection (16
bytes)

From server_initial_secret:
  ■■ HKDF-Expand-Label(Label="quic key", Length=16)
    = 0xcf3a5331653c364c88f0f379b6067e37 // AES-128 key
  ■■ HKDF-Expand-Label(Label="quic iv", Length=12)
    = 0x0ac1493ca1905853b0bba03e // IV
  ■■ HKDF-Expand-Label(Label="quic hp", Length=16)
    = 0xc206b8d9b9f0f37644430b490eeaa314 // HP key
```

Total: from one 8-byte DCID, we derived 6 keys (3 per direction). Each direction has an AEAD key (for encrypting/decrypting payload), an IV (combined with packet number to form the nonce), and an HP key (for obscuring the packet number in the header).

4. How AEAD Encryption Works in QUIC

QUIC uses AEAD (Authenticated Encryption with Associated Data) -- specifically AES-128-GCM or ChaCha20-Poly1305. AEAD encrypts the payload AND authenticates both the payload and the header. Any tampering is detected.

4.1 Encryption (Sender Side)

```
Inputs:
  key   = AEAD key (16 bytes for AES-128-GCM)
  nonce = IV XOR packet_number (12 bytes)
  aad   = QUIC header bytes (authenticated but NOT encrypted)
  plain = payload to encrypt

Output:
  cipher = AES-128-GCM-Encrypt(key, nonce, aad, plain)
         = encrypted_payload (same length as plain)
         + authentication_tag (16 bytes)

Example:
  key   = 0x1f369613dd76d5467730efcbe3b1a22d
  IV    = 0xfa044b2f42a3fd3b46fb255c
  PN    = 0x00000002 (packet number 2)
  nonce = IV XOR PN = 0xfa044b2f42a3fd3b46fb255e
                                ^^^^ (last bytes changed by XOR)

  aad   = [QUIC short header bytes...]
  plain = [CRYPTO frame with ClientHello...]
  [REDACTED]
  cipher = [encrypted bytes...] + [16-byte auth tag]
```

The nonce changes for every packet (because packet number changes). This ensures that even identical payloads produce different ciphertext. The AEAD key stays the same for the entire epoch.

4.2 Header Protection (HP)

After AEAD encryption, the packet number in the header is still visible. QUIC applies a second layer: Header Protection. This obscures the packet number so observers can't easily track packet sequences.

```
Header Protection:
1. Sample 16 bytes from the ciphertext (at a fixed offset)
2. Compute mask = AES-ECB(hp_key, sample) [first 5 bytes used]
3. XOR the header flags with mask[0] (partially)
4. XOR the packet number bytes with mask[1..5]

Result: packet number is obscured in the wire format
Receiver reverses the process using the same HP key
```

4.3 Summary: Three Keys, Three Jobs

Key	Size (AES-128)	Purpose	Changes When?
AEAD key	16 bytes	Encrypt/decrypt packet payload + auth tag	Only at key update (epoch change)

Key	Size (AES-128)	Purpose	Changes When?
IV	12 bytes	Combined with packet number to form unique nonce	Only at key update
HP key	16 bytes	Obscure packet number in header	Only at key update

5. Phase 2: Handshake Keys (Epoch 2)

After Initial packets are exchanged, the TLS handshake happens inside them. The client and server perform an Elliptic Curve Diffie-Hellman key exchange (ECDHE). This produces a shared secret that no one else knows.

Step 1: Key Exchange (X25519)

```
Client generates:
  client_private = random 32 bytes (NEVER leaves client)
  client_public  = X25519_basepoint(client_private)

0x358072d6365880d1aeea329adf9121383851ed21a28e3b75e965d0d2cd166254

Server generates:
  server_private = random 32 bytes (NEVER leaves server)
  server_public  = X25519_basepoint(server_private)

0x9fd7ad6dcff4298dd3f96d5b1b2af910a0535b1488d7f8fabb349a982880b615

Exchange: client sends client_public in ClientHello key_share
          server sends server_public in ServerHello key_share

Both compute INDEPENDENTLY:
  shared_secret = X25519(my_private, their_public)

Client: X25519(client_private, server_public) = 0xdf4a291baa1eb7cf...
Server: X25519(server_private, client_public) = 0xdf4a291baa1eb7cf...
                                             ^^^^^^^^^^^^^^^^^^^^^
                                             SAME VALUE! (32 bytes)
```

This is the magic of Diffie-Hellman: both sides compute the same shared secret without ever transmitting it. An observer who sees both public keys CANNOT compute the shared secret (this would require solving the discrete log problem on Curve25519).

Step 2: Derive Handshake Secret

```
// First, derive an intermediate "early secret" (no PSK, so IKM = 0)
early_secret = HKDF-Extract(salt=0, IKM=0x00...00)

// Derive a salt for the next extraction
derived_secret = HKDF-Expand-Label(early_secret, "derived", "", 32)

// Extract handshake secret using the ECDHE shared secret
handshake_secret = HKDF-Extract(
  salt = derived_secret,
  IKM  = shared_secret    // from Diffie-Hellman above
)
= 0x[32 bytes -- this is the handshake secret]
```

Step 3: Derive Handshake Traffic Secrets


```
// The "Context" here is the hash of all handshake messages so far
// (ClientHello + ServerHello). This binds the keys to THIS specific handshake.

transcript_hash = SHA-256(ClientHello || ServerHello)

client_hs_secret = HKDF-Expand-Label(
    handshake_secret, "c hs traffic", transcript_hash, 32
)

server_hs_secret = HKDF-Expand-Label(
    handshake_secret, "s hs traffic", transcript_hash, 32
)

// From each, derive AEAD key + IV + HP key (same as Initial phase)
// These encrypt the rest of the handshake (Certificate, Finished, etc.)
```

Handshake keys are only used for a few packets (server's Certificate, CertificateVerify, Finished; client's Finished). After that, both sides switch to Application keys.

6. Phase 3: Application Keys (Epoch 3) -- What We Export

This is the most important phase for our migration. Application keys protect ALL data after the handshake -- every HTTP/3 request, every response, every stream. These are the keys we export to the preferred server.

The application traffic secrets are what we transfer in our 445-byte migration state. Everything in this section is critical to understanding the migration.

Step 1: Derive Master Secret

```
// Continue the key schedule from the handshake secret
derived_secret_2 = HKDF-Expand-Label(handshake_secret, "derived", "", 32)

// Master secret -- no more DH input (IKM = 0)
master_secret = HKDF-Extract(
    salt = derived_secret_2,
    IKM = 0x00...00 (32 zero bytes)
)
= 0x[32 bytes]
```

Step 2: Derive Application Traffic Secrets

```
// Transcript now includes the FULL handshake
transcript_hash = SHA-256(ClientHello || ... || server Finished)

client_app_secret_0 = HKDF-Expand-Label(
    master_secret,
    "c ap traffic",          // "client application traffic"
    transcript_hash,
    32                      // 32 bytes for SHA-256 based ciphers
)
= 0x9e40646ce79a7f9dc05af8889bce6552875afa0b06df0087f792ebb7c17504a5
  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  THIS IS THE CLIENT APP SECRET -- 32 bytes we export for migration

server_app_secret_0 = HKDF-Expand-Label(
    master_secret,
    "s ap traffic",          // "server application traffic"
    transcript_hash,
    32
)
= 0xa11af9f05531f856ad47116b45a950328204b4f44bfb6b3a4b4f1f3fcb631643
  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  THIS IS THE SERVER APP SECRET -- 32 bytes we export for migration
```

These two 32-byte secrets are the crown jewels. With them, you can derive every key needed to decrypt and encrypt all application data for this connection. This is what makes server migration possible.

Step 3: Derive AEAD Key, IV, and HP Key

From client_app_secret_0 (for decrypting client→server packets):

```
client_app_key = HKDF-Expand-Label(client_app_secret_0, "quic key", "", 16)
                = 0xe010a295f0c2864f186b2a7e8fdc9ed7 (16-byte AES key)
```

```
client_app_iv  = HKDF-Expand-Label(client_app_secret_0, "quic iv", "", 12)
                = 0x0ae7b6b932bc27d786f4bc2b (12-byte IV)
```

```
client_app_hp  = HKDF-Expand-Label(client_app_secret_0, "quic hp", "", 16)
                = 0x25a282b9e82f06f21f488917a4fc8f1b (16-byte HP key)
```

From server_app_secret_0 (for encrypting server→client packets):

```
server_app_key = HKDF-Expand-Label(server_app_secret_0, "quic key", "", 16)
                = 0xfd8c7da9de1b2da4d2ef9fd5188922d0 (16-byte AES key)
```

```
server_app_iv  = HKDF-Expand-Label(server_app_secret_0, "quic iv", "", 12)
                = 0x02f6180e4f4aa456d7e8a389 (12-byte IV)
```

```
server_app_hp  = HKDF-Expand-Label(server_app_secret_0, "quic hp", "", 16)
                = 0xb7f6f021453e43b0b2c25989d0a31207 (16-byte HP key)
```

Critical insight: We do NOT export these 6 derived keys. We export the 2 traffic secrets (32 bytes each). The preferred server re-runs HKDF-Expand-Label with the same labels and gets the EXACT same keys. HKDF is deterministic -- same input always produces same output.

7. Key Update: How Keys Evolve

TLS 1.3 supports key updates: either side can request new keys at any time. This provides forward secrecy even within a single connection -- if keys from epoch N are compromised, epochs N+1, N+2... remain secure.

Key Update derivation:

```
client_app_secret_1 = HKDF-Expand-Label(
  client_app_secret_0,    // current secret
  "quic ku",             // "QUIC key update"
  "",                    // 
  32
)
= 0x[new 32-byte secret]
■■■ derive new AEAD key, IV, HP key (same process as before)

client_app_secret_2 = HKDF-Expand-Label(client_app_secret_1, "quic ku", "", 32)
client_app_secret_3 = HKDF-Expand-Label(client_app_secret_2, "quic ku", "", 32)
... and so on
```

Each generation:

- Epoch increments (3 → 4 → 5 → ...)
- Old keys are discarded
- Cannot go backwards (can't derive secret_0 from secret_1)

This is why we export TWO secrets per direction: the current secret (to derive current keys) and the next secret (to handle the next key update). If a key update happens right after migration, the preferred server is ready.

8. What We Export: The 445 Bytes

Now we can understand exactly what's in our migration state and why each field is necessary.

8.1 Crypto State (198 bytes)

Field	Size	Example Value	Why Needed
Write secret (server→client)	2+32 = 34 bytes	0x0020 a11af9f05531f856...	Re-derive AEAD key + IV + HP for encrypting responses
Write next secret	2+32 = 34 bytes	0x0020 7b4f2e8c9a1d3f5e...	Handle key update after migration
Write cipher suite	2 bytes	0x1301 (TLS_AES_128_GCM_SHA256)	Which algorithm to use for key derivation
Write direction	1 byte	0x01 (Write)	Identifies this as the write direction
Write epoch	4 bytes	0x00000003 (epoch 3 = first app data)	Which key generation we're on
Write PN range	24 bytes	start=0, end=15, min=0	Continue packet numbering without gaps/replays

Field	Size	Example Value	Why Needed
Read secret (client→server)	34 bytes	0x0020 9e40646ce79a7f9d...	Re-derive AEAD key for decrypting client packets
Read next secret	34 bytes	0x0020 3c8b1d4e7f2a9085...	Handle key update from client side
Read cipher + dir + epoch + PNs	29 bytes	(same structure as write)	Mirror of write crypto for read direction

8.2 Connection IDs (194 bytes)

```
Local CIDs (what we issued to the client -- 8 CIDs typical):
Count: 0x0008
CID 0: seqno=0x0000000000000000 len=0x000A data=0x7f8a3c1b9e2d4f06a5c8
CID 1: seqno=0x0000000000000001 len=0x000A data=0x3b7e9d2f1c8a4506e7b3
... (6 more)
```

Why: The client uses these CIDs to address packets to us.
Preferred server must recognize them as valid.

```
Remote CIDs (what the client issued to us -- typically 1):
Count: 0x0001
CID 0: seqno=0x0000000000000000 len=0x000A data=0x2c5f8b1a7d3e9046c8a2
```

Why: We use this CID in our packet headers so the client recognizes our packets as belonging to this connection.

8.3 Client Address (7 bytes for IPv4)

```
Address type: 0x04 (IPv4)
IP address:   0x8DD9A87F = 141.217.168.127
Port:        0xC350      = 50000
```

Why: The preferred server needs to know WHERE to send packets.
Without this, it doesn't know the client's IP:port.

8.4 Complete Binary Layout

9. Import on the Preferred Server: Reconstructing Keys

The preferred server receives 445 bytes via TCP/Redis/HTTP. Here's exactly what happens to reconstruct a fully functional QUIC connection:

Step 1: Deserialize the binary state

```
Parse 445 bytes:
  ■■ Verify magic = "MIGR"
  ■■ Read QUIC version
  ■■ Extract write_secret_bytes = [32 bytes]
  ■■ Extract write_next_secret_bytes = [32 bytes]
  ■■ Extract read_secret_bytes = [32 bytes]
  ■■ Extract read_next_secret_bytes = [32 bytes]
  ■■ Extract CIDs, client address, etc.
```

Step 2: Reconstruct NSS SymKey objects from raw bytes

```
// Raw bytes → NSS internal key object
write_secret = hkdf::import_key(TLS_VERSION_1_3, &write_secret_bytes)

// What import_key does internally:
// 1. Allocates an NSS PK11SymKey structure
// 2. Copies the 32 raw bytes into NSS's key storage
// 3. Tags it as an HKDF key for TLS 1.3
// 4. Returns a SymKey handle

// This SymKey is now IDENTICAL to the one the primary server had
// Same bytes = same key = same derivation results
```

Step 3: Re-derive AEAD and HP keys (deterministic!)

```
// This is the critical step -- HKDF is deterministic

// On the PRIMARY server during handshake:
HKDF-Expand-Label(secret_0xa11af9f0..., "quic key", 16)
= 0xfd8c7da9de1b2da4d2ef9fd5188922d0

// On the PREFERRED server during import:
HKDF-Expand-Label(secret_0xa11af9f0..., "quic key", 16)
= 0xfd8c7da9de1b2da4d2ef9fd5188922d0 ← IDENTICAL!

// Same for IV:
Both: HKDF-Expand-Label(secret, "quic iv", 12) = 0x02f6180e4f4aa456d7e8a389

// Same for HP:
Both: HKDF-Expand-Label(secret, "quic hp", 16) =
0xb7f6f021453e43b0b2c25989d0a31207

// The preferred server now has byte-identical AEAD and HP keys
// It CAN decrypt packets from the client
// It CAN encrypt packets back to the client
```

This is the fundamental principle: HKDF is a deterministic function. Same secret + same label + same length = ALWAYS the same output. The preferred server never did the TLS handshake, never saw the

Diffie-Hellman exchange, never verified the certificate -- but it has identical encryption keys.

Step 4: Set up the connection state

```
On the preferred server:
conn.state = State::Confirmed    // Skip handshake entirely
conn.version = QUIC_V1
conn.write_crypto = re-derived write keys
conn.read_crypto = re-derived read keys
conn.local_cids = [8 imported CIDs]
conn.remote_cids = [1 imported CID]
conn.path = client_addr:port ↔ local_addr:port
conn.next_pn = used_pn_end      // Continue packet numbering
```

The connection object looks as if the handshake happened locally.

The preferred server can now:

1. Decrypt PATH_CHALLENGE from client (using read keys)
2. Encrypt PATH_RESPONSE back to client (using write keys)
3. Handle all subsequent data exchange

10. End-to-End Example: What Actually Happens

Let's trace a complete connection from Firefox connecting to our primary server through migration to the preferred server, showing the keys at each step.

Step 1: Firefox sends Initial packet

```
Firefox → Primary (141.217.168.152:4433)

QUIC Long Header:
  Version: 0x00000001 (QUIC v1)
  DCID: 0x7f8a3c1b9e2d4f06 (random, 8 bytes)
  SCID: 0xa5c83b7e (Firefox's CID, 4 bytes)

Encrypted with Initial keys derived from DCID:
  initial_secret = HKDF-Extract(QUIC_v1_salt, 0x7f8a3c1b9e2d4f06)
  client_initial_secret = HKDF-Expand-Label(initial_secret, "client in")
  client_key = HKDF-Expand-Label(client_initial_secret, "quic key")

Payload (decryptable by anyone!):
  CRYPTO frame containing TLS ClientHello:
    - SNI: 141.217.168.152
    - ALPN: h3
    - Key Share: X25519 public key (32 bytes)
    - Supported Versions: TLS 1.3
```

Step 2: Primary server responds with handshake + preferred_address

```
Primary → Firefox

ServerHello (in Initial packet, Initial keys):
  - Key Share: server's X25519 public key
  - Cipher Suite: TLS_AES_128_GCM_SHA256 (0x1301)

Both sides now compute:
  shared_secret = X25519(my_private, their_public)
  → handshake_secret → handshake keys

EncryptedExtensions (in Handshake packet, Handshake keys):
  QUIC Transport Parameters:
    - preferred_address: 141.217.168.143:4433 ← PREFERRED SERVER!
    - preferred_address_cid: 0x3b7e9d2f1c...
    - max_idle_timeout: 30000
    - initial_max_data: 1048576
    - (other transport params...)

Certificate + CertificateVerify + Finished (Handshake keys)

After Finished exchange:
  → master_secret
  → client_app_secret = 0x9e40646ce79a7f9d...
  → server_app_secret = 0xa11af9f05531f856...
  Switch to Application keys (Epoch 3)
```

Step 3: Primary exports migration state

Primary server calls `export_migration_state()`:

```
write_crypto.export_for_migration():
    secret_bytes = self.secret.key_data()
                  = [0xa1, 0x1a, 0xf9, 0xf0, 0x55, 0x31, ... ] (32 bytes)
    next_secret  = self.next_secret.key_data()
                  = [0x7b, 0x4f, 0x2e, 0x8c, 0x9a, 0x1d, ... ] (32 bytes)
    cipher       = 0x1301 (TLS_AES_128_GCM_SHA256)
    epoch        = 3
    used_pn      = 0..15 (sent packets 0 through 14)

read_crypto.export_for_migration():
    secret_bytes = [0x9e, 0x40, 0x64, 0x6c, 0xe7, 0x9a, ... ] (32 bytes)
    next_secret  = [0x3c, 0x8b, 0x1d, 0x4e, 0x7f, 0x2a, ... ] (32 bytes)
    used_pn      = 0..3 (received packets 0 through 2)
```

Serialize to 445 bytes with "MIGR" header

Step 4: State transfer (TCP push to preferred server)

Primary (141.217.168.152) TCP:9999 Preferred (141.217.168.143)

Send: [4 bytes length] [445 bytes migration state]

Time: < 1ms on LAN

The 445 bytes contain:

- Bytes 0-7: MIGR + version
- Bytes 8-105: Write crypto (secret + next + cipher + epoch + PNs)
- Bytes 106-203: Read crypto (same structure)
- Bytes 204-365: Local CIDs (8 CIDs)
- Bytes 366-385: Remote CIDs (1 CID)
- Bytes 386-392: Client addr (IPv4 + port)

Step 5: Preferred server imports state and rebuilds keys

```
Preferred server receives 445 bytes, calls new_from_migration():

// Reconstruct write keys (for server→client)
write_secret = hkdf::import_key(TLS_1_3, [0xa1, 0x1a, 0xf9, ...])
write_aead = Aead::new(TLS_1_3, AES_128_GCM, write_secret, "quic ")
internally:
    key = HKDF-Expand-Label(write_secret, "quic key", 16)
        = 0xfd8c7da9de1b2da4d2ef9fd5188922d0 ← IDENTICAL to primary
    iv = HKDF-Expand-Label(write_secret, "quic iv", 12)
        = 0x02f6180e4f4aa456d7e8a389 ← IDENTICAL to primary
write_hp = hp::Key::extract(TLS_1_3, AES_128_GCM, write_secret, "quic hp")
        = 0xb7f6f021453e43b0b2c25989d0a31207 ← IDENTICAL to primary

// Reconstruct read keys (for client→server)
read_secret = hkdf::import_key(TLS_1_3, [0x9e, 0x40, 0x64, ...])
read_aead = Aead::new(...)
read_hp = hp::Key::extract(...)
// All IDENTICAL to what the primary server had

// Set up connection
conn.state = Confirmed
conn.path = 141.217.168.127:50000 ↔ 0.0.0.0:4433
conn.local_cids = [8 CIDs from migration state]
conn.remote_cids = [1 CID from migration state]

// Ready to handle packets!
```

Step 6: Firefox sends PATH_CHALLENGE (migration validation)

```
Firefox → Preferred (141.217.168.143:4433)

Firefox follows RFC 9000 Section 9.6:
    "When the client receives preferred_address, it SHOULD initiate
    path validation to the preferred address."

QUIC Short Header:
    DCID: 0x3b7e9d2f1c8a4506e7b3 (one of our local CIDs)
    Packet Number: 3 (encrypted with HP key)

Encrypted with client_app_key + nonce(IV XOR PN=3):
    Payload: PATH_CHALLENGE frame
        data: 0x4a8b2c9d1e7f3a06 (random 8 bytes)

Preferred server decrypts:
    1. Remove HP: use read_hp_key to recover packet number
    2. Compute nonce = read_iv XOR 3
    3. AEAD-Decrypt(read_key, nonce, header, ciphertext)
    4. Parse: PATH_CHALLENGE { data: 0x4a8b2c9d1e7f3a06 }
    ✓ Decryption succeeds! Keys are correct.
```

Step 7: Preferred server sends PATH_RESPONSE

Preferred → Firefox (141.217.168.127:50000)

QUIC Short Header:

DCID: 0xa5c83b7e (Firefox's CID from remote_cids)

Packet Number: 15 (continuing from primary's used_pn_end)

Encrypted with server_app_key + nonce(IV XOR PN=15):

Payload: PATH_RESPONSE frame

data: 0x4a8b2c9d1e7f3a06 (echo back the SAME 8 bytes)

Firefox receives and decrypts:

1. Uses its server_app_key to decrypt
 2. Sees PATH_RESPONSE with matching data
 3. Path validation SUCCEEDS
 4. Firefox switches to using 141.217.168.143:4433
- ✓ Migration complete! Firefox is unaware a different machine responded.

The entire migration is invisible to Firefox. It received a PATH_RESPONSE encrypted with keys that only the legitimate server should know. From Firefox's perspective, the server simply moved to a new address (as advertised in preferred_address). It has no way to know that a completely different physical machine generated that response.

11. Why Export the Secret, Not the Derived Keys?

A natural question: why not just export the 6 derived keys (2 AEAD + 2 IV + 2 HP = 88 bytes) instead of the 2 secrets (64 bytes)?

Approach	Size	Pro	Con
Export 2 secrets	64 bytes (+ next secrets = 128)	Supports key update on preferred server	Must re-derive 6 keys (fast, ~microseconds)
Export 6 derived keys	88 bytes	No re-derivation needed	Cannot do key update! No secret to derive next generation

We export the secrets because the preferred server needs to support key updates. If the client requests a key update after migration, the preferred server must derive new keys from the current secret. Without the secret, it's stuck on the current keys forever.

Additionally, NSS (the crypto library) constructs AEAD and HP objects from the secret during initialization. There's no API to import raw AEAD keys -- you must provide the secret and let NSS derive the keys internally.

12. Security Implications of Secret Export

Exporting TLS traffic secrets has serious security implications. Understanding these is essential for evaluating the migration approach.

- **The 32-byte secret IS the connection:** Anyone with this secret can decrypt all past and future packets (until key update). There's no additional authentication.
- **Forward secrecy is preserved up to the export point:** The handshake used ECDHE, so compromising the server's long-term certificate key doesn't reveal the traffic secret. But the traffic secret itself must be protected during transfer.
- **Transfer channel security is critical:** The TCP/Redis/HTTP channel carrying the 445 bytes must be secured. In our demo (same LAN), we use plaintext TCP. In production, this **MUST** be encrypted (mTLS, TLS, encrypted Redis).
- **Our 5 backends have different security profiles:**
 - TCP Push: secret transits the network (encrypt the channel!)
 - Redis KV: secret stored temporarily in Redis (encrypt at rest!)
 - Redis Pub/Sub: secret transits Redis (encrypt the channel!)
 - HTTP Pull: secret stays in primary's memory until pulled (best security!)
 - File: secret written to disk (worst security!)
- **QUIC-Exfil attack:** This same mechanism enables data exfiltration. A malicious server can migrate connections to an attacker-controlled machine. The client can't distinguish this from legitimate migration.

13. Summary: The Complete Key Lifecycle

CLIENT	PRIMARY SERVER	PREFERRED SERVER
--- Initial (DCID keys) ----->		
<-- Initial + ServerHello -----		
[ECDHE shared secret computed]		
--- Handshake (HS keys) ----->		
<-- Handshake (cert, finished) ----		
[Master secret derived]		
[App traffic secrets derived]		
<== Application data (App keys) ==>		
	-- [445 bytes via TCP]----->	
	(2 secrets + CIDs + addr)	
	[Re-derive keys]	
	[IDENTICAL keys]	
--- PATH_CHALLENGE (App keys) ----- ----->		
<-- PATH_RESPONSE (App keys) ----- -----		
<===== Application data (App keys) =====>		
Firefox thinks it's the same server, just at a new address		

The entire security of QUIC rests on the traffic secrets derived from the TLS 1.3 handshake. By exporting just 64 bytes of secrets (+ 381 bytes of connection metadata), we transfer the complete ability to communicate with the client. The preferred server reconstructs byte-identical keys through deterministic HKDF derivation. The client cannot tell the difference.